A Beacon, a Schedule, and a Little APRS Magic

*A lightweight Python tool for putting fixed stations on the APRS map – by **Erwin, PA3EFR***

Ever wished you could put an APRS station on the map without leaving a radio running on the desk, or arrange for a beacon to appear only during a particular event? This small Python script takes a different route: it sends position reports directly to APRS-IS, using one or more fixed locations. Think of it as a programmable map pin with a callsign—and a diary.

More than one pin on the map

The script is designed for fixed-position APRS beacons. You enter a callsign, coordinates, a short comment, a symbol, and a transmission interval. You can then add another beacon, and another, each with its own settings. When you finish configuring them, the program opens an aprs.fi browser tab for every callsign, giving you an easy way to watch what happens.

That makes it handy for demonstrations, temporary event locations, or simply experimenting with how different fixed positions appear on the APRS map. The positions are configured by the operator; this is not GPS tracking of a moving station.

Give each beacon its own timetable

The useful trick is scheduling. Each beacon can have its own start date and time, plus an optional end date and time. Leave the start time blank and the script starts now if the chosen date is today; for a future date, it starts at midnight. Leave both end fields blank and the beacon keeps going until you stop the program with Ctrl+C.

In other words, you can prepare the whole line-up in advance. One beacon can start later, another can finish before the first coffee break, and a third can continue indefinitely. The script checks each beacon independently, so they do not all have to share the same schedule.

Coordinates, symbols, and short messages

Coordinates can be entered as decimal latitude and longitude (for example, “52.0705, 4.3007”) or in degrees with N/S and E/W indicators. The script checks that the values fall within valid latitude and longitude ranges, which helps catch the classic swapped-coordinates mistake before it reaches the map.

You can choose an APRS symbol too: a house for a fixed station, an antenna, a digipeater, a weather station, a car, a bicycle, a walker, a motorcycle, or a truck are among the handy examples listed by the program. If you are unsure of the symbol code, enter “?” to see a short menu and a pointer to the full symbol list. The default is the scout symbol.

The comment field is limited to 43 characters. If you type a longer comment, the script trims it to fit rather than making a fuss. Callsigns are checked for a plausible format, and dates and times accept several familiar formats. Invalid input gets another prompt instead of immediately derailing the session.

A little housekeeping behind the scenes

To send packets to APRS-IS, the script uses the Python library aprslib and connects to the configured APRS-IS server on TCP port 14580. It builds an uncompressed APRS position packet from the configured coordinates, symbol, and comment, then sends it over the internet. This is an APRS-IS submission tool, not a program that keys a VHF transmitter or replaces an RF gateway.

For authentication, it looks for callsign/passcode pairs in a local file named “APRS_Codes.txt”. If a callsign is missing, it asks the operator to provide a passcode. Once a passcode has been entered, the script remembers it during that run, so it does not keep asking for the same callsign. Keep that file private and make sure you use the appropriate passcode for your station.

Keep an eye on the log

The console shows a summary of each beacon’s callsign, coordinates, APRS information, symbol, interval, start time, and end time. During operation, every send attempt is logged as successful or failed, and the script keeps a simple count of successful submissions versus total attempts. At the end, it prints a final tally and the aprs.fi links again. That

is useful when something goes wrong: the log gives you a first clue instead of leaving you to guess whether your beacon has gone quiet.

A sensible interval—and a sensible reminder

The default interval is ten minutes. The script allows intervals as short as one minute, but warns that at least ten minutes is recommended for a fixed beacon. That warning is worth listening to: just because a computer can send frequent position reports does not mean it should. Use an interval that suits the purpose, and be considerate of APRS-IS and the wider APRS community.

What you need to try it

You will need Python, the aprslib package, internet access, and the local passcode file—or a passcode ready to enter when prompted. Start the script in a terminal, answer the questions for each beacon, check the overview carefully, and then let it run. A scheduled start in the future is handled by the program; a beacon whose start time is already past begins immediately.

One final practical detail: the script opens an aprs.fi tab for each configured callsign, but the browser tabs are for monitoring, not for sending. The actual position reports are submitted by the Python program.

73, and may all your map pins land exactly where you intended.



Three fixed beacons plotted via the script.

Repository: https://github.com/PA3EFR/aprs_mutli-beacon-automation